



Darshan
UNIVERSITY

Data Mining

Lab - 1

Kavyam Joshi | 23010101114 |
02/06/2025

Introduction to Pandas Library Function:

Step-1 Import the pandas Libraries

```
In [1]: import pandas as pd
```

Step-2 Import the dataset from this:.....

```
In [ ]:
```

Step-3 Read csv or excel File

```
In [2]: df=pd.read_csv('Titanic.csv')  
df
```

```
Out[2]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

Step-4 Print Data from csv or excel File

```
In [3]: df
```

Out[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

Step-5 See the First 10 Rows

In [4]: `df.head(10)`

Out[4]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736

Step-6 See the Last 10 Rows

```
In [5]: df.tail(10)
```

Out[5]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	T
881	882	0	3	Markun, Mr. Johann	male	33.0	0	0	34
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./S(3
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTO39
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	38
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	21
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	11
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C.
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	11
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	37

Step-7 Data type of each columns

In [9]: `df.dtypes`

```
Out[9]: PassengerId      int64
Survived      int64
Pclass        int64
Name          object
Sex           object
Age           float64
SibSp         int64
Parch         int64
Ticket        object
Fare          float64
Cabin         object
Embarked      object
dtype: object
```

Step-8 Display Summary Information

```
In [6]: df.describe()
```

```
Out[6]:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000

```
In [4]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column             Non-Null Count  Dtype
---  -
0   PassengerId         891 non-null    int64
1   Survived            891 non-null    int64
2   Pclass              891 non-null    int64
3   Name                891 non-null    object
4   Sex                 891 non-null    object
5   Age                 714 non-null    float64
6   SibSp               891 non-null    int64
7   Parch              891 non-null    int64
8   Ticket              891 non-null    object
9   Fare                891 non-null    float64
10  Cabin               204 non-null    object
11  Embarked            889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Step-9 Access a specific column

```
In [5]: df['Age']
```

```

Out[5]: 0      22.0
        1      38.0
        2      26.0
        3      35.0
        4      35.0
        ...
       886     27.0
       887     19.0
       888      NaN
       889     26.0
       890     32.0
        Name: Age, Length: 891, dtype: float64

```

```
In [11]: df.shape
```

```
Out[11]: (891, 12)
```

```
In [12]: df.shape[0]
```

```
Out[12]: 891
```

```
In [13]: df.shape[1]
```

```
Out[13]: 12
```

Step-10 Access rows by their integer location

```
In [6]: df.iloc[100:200]
```

```
Out[6]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
100	101	0	3	Petranec, Miss. Matilda	female	28.0	0	0	34980
101	102	0	3	Petroff, Mr. Pastcho ("Pentcho")	male	NaN	0	0	34980
102	103	0	1	White, Mr. Richard Frasar	male	21.0	0	1	35040
103	104	0	3	Johansson, Mr. Gustaf Joel	male	33.0	0	0	70651
104	105	0	3	Gustafsson, Mr. Anders Vilhelm	male	37.0	2	0	31012
...	...	...	...	...	...	...	...	...	...
195	196	1	1	Lurette, Miss. Elise	female	58.0	0	0	17463
196	197	0	3	Mernagh, Mr. Robert	male	NaN	0	0	36851
197	198	0	3	Olsen, Mr. Karl Siegwart Andreas	male	42.0	0	1	41706
198	199	1	3	Madigan, Miss. Margaret "Maggie"	female	NaN	0	0	37031
199	200	0	2	Yrois, Miss. Henriette ("Mrs Harbeck")	female	24.0	0	0	24160

100 rows × 12 columns

Step-11 Delete a specific Column

```
In [17]: df.drop("NewCol",axis=1,inplace=True)
```

```
In [10]: df
```

Out[10]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
	0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
	2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
	4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
	...	...	...	...	...	...	...	...	...	
	886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
	888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 66
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
	890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 11 columns

Step-12 Create a new Column

```
In [21]: df["isCabin"] = ~df["Cabin"].isnull()
```

In [22]: df

Out[22]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211536
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38.0	1	0	17597
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/OJ 310128
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373451
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112052
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W/ 6616
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370371

891 rows × 12 columns

Step-13 Perform Condition Selection on DataFrame

```
In [23]: df[df['Pclass']==3]
```

Out[23]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211506343
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/OJ 3101282
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373451
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330814
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349021
...	...	...	...	...	...	...	...	...	...
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	759017
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OJ 392039
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W/ 661634
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376

491 rows x 12 columns

Step-14 Compute the sum of value

```
In [24]: df["Fare"].sum()
```

```
Out[24]: 28693.9493
```

Step-15 Compute the mean of value

```
In [25]: df["Fare"].mean()
```

```
Out[25]: 32.204207968574636
```

Step-16 Count non-null value (column)

```
In [37]: (df.isnull()).sum()
```

```
Out[37]: PassengerId      0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
isCabin       0
dtype: int64
```

```
In [33]: (~df.isnull()).sum()
```

```
Out[33]: PassengerId      -1
Survived      -1
Pclass        -1
Name          -1
Sex           -1
Age          -178
SibSp         -1
Parch         -1
Ticket        -1
Fare          -1
Cabin       -688
isCabin       -1
dtype: int64
```

Step-17 Find Minimum or Maximum values

```
In [35]: df["Fare"].max()
```

```
Out[35]: 512.3292
```

```
In [36]: df["Fare"].min()
```

```
Out[36]: 0.0
```



Data Mining

Lab - 2

Kavyam Joshi | 23010101114 |
09/06/2025

```
# Data Mining - Lab - 2
```

```
# Numpy & Perform Data Exploration with Pandas
```

Numpy

1. NumPy (Numerical Python) is a powerful open-source library in Python used for numerical and scientific computing.
2. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them efficiently.
3. NumPy is highly optimized and written in C, making it much faster than using regular Python lists for numerical operations.
4. It serves as the foundation for many other Python libraries in data science and machine learning, like pandas, TensorFlow, and scikit-learn.
5. With features like broadcasting, vectorization, and integration with C/C++ code, NumPy allows for cleaner and faster code in numerical computations.

Step 1. Import the Numpy library

In []:

```
In [51]: import numpy as np
```

Step 2. Create a 1D array of numbers

```
In [2]: a=np.arange(11)
print(a)
print(type(a))
```

```
[ 0  1  2  3  4  5  6  7  8  9 10]
<class 'numpy.ndarray'>
```

```
In [5]: a=np.arange(2,9)
a
#print(type(a))
```

```
Out[5]: array([2, 3, 4, 5, 6, 7, 8])
```

Step 3. Reshape 1D to 2D Array

```
In [38]: a=np.arange(12).reshape(3,4)
a
```

```
Out[38]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11]])
```

Step 4. Create a Linspace array

```
In [11]: np.linspace(0,5,20)
```

```
Out[11]: array([0.          , 0.26315789, 0.52631579, 0.78947368, 1.05263158,
                1.31578947, 1.57894737, 1.84210526, 2.10526316, 2.36842105,
                2.63157895, 2.89473684, 3.15789474, 3.42105263, 3.68421053,
                3.94736842, 4.21052632, 4.47368421, 4.73684211, 5.          ])
```

Step 5. Create a Random Numbered Array

```
In [12]: np.random.rand(2)
```

```
Out[12]: array([0.77556139, 0.22346946])
```

```
In [15]: np.random.rand(2,4)
```



```
Out[15]: array([[0.10567208, 0.80100079, 0.7450214 , 0.80170639],
               [0.39946002, 0.92583548, 0.13977874, 0.23842287]])
```

Step 6. Create a Random Integer Array

```
In [16]: np.random.randint(1,100,size=10)
```

```
Out[16]: array([74, 34, 93, 30, 47, 98, 42, 15, 42, 55])
```

```
In [19]: np.random.randint(1,100,size=(5,2))
```

```
Out[19]: array([[51, 54],
               [53, 99],
               [48, 19],
               [54, 92],
               [89, 29]])
```

Step 7. Create a 1D Array and get Max,Min,ArgMax,ArgMin

```
In [29]: arr = np.random.randint(1,100,size=10)
arr
```

```
Out[29]: array([57, 49, 99,  5, 28, 33, 26, 78, 54, 60])
```

```
In [30]: arr.max()
```

```
Out[30]: 99
```

```
In [31]: arr.min()
```

```
Out[31]: 5
```

```
In [32]: arr.argmax()
```

```
Out[32]: 2
```

```
In [33]: arr.argmin()
```

```
Out[33]: 3
```

Step 8. Indexing in 1D Array

```
In [34]: arr[8]
```

```
Out[34]: 54
```

```
In [35]: arr[1:5]
```

```
Out[35]: array([49, 99,  5, 28])
```

Step 9. Indexing in 2D Array

```
In [39]: arr2=arr.reshape(2,5)  
arr2
```

```
Out[39]: array([[57, 49, 99,  5, 28],  
               [33, 26, 78, 54, 60]])
```

```
In [40]: arr2[0]
```

```
Out[40]: array([57, 49, 99,  5, 28])
```

```
In [44]: arr2[0][1]  
arr2[0,1]
```

```
Out[44]: 49
```

```
In [45]: arr2[:,2:]
```

```
Out[45]: array([[99,  5, 28]])
```

```
In [55]: roll= np.arange(1,16).reshape(3,5)  
roll
```

```
Out[55]: array([[ 1,  2,  3,  4,  5],  
               [ 6,  7,  8,  9, 10],  
               [11, 12, 13, 14, 15]])
```

```
In [56]: f1=roll[1:]  
f1
```

```
Out[56]: array([[ 6,  7,  8,  9, 10],  
               [11, 12, 13, 14, 15]])
```

```
In [58]: f1=roll[:,2:]  
f1
```

```
Out[58]: array([[ 1,  2,  3,  4,  5],  
               [11, 12, 13, 14, 15]])
```

```
In [60]: f1=roll[1:,2::]  
f1
```

```
Out[60]: array([[ 8,  9, 10],  
               [13, 14, 15]])
```

Step 10. Conditional Selection

```
In [61]: arr[arr>4]
```

```
Out[61]: array([57, 49, 99,  5, 28, 33, 26, 78, 54, 60])
```

```
In [62]: arr2[arr2>2]
```

```
Out[62]: array([57, 49, 99,  5, 28, 33, 26, 78, 54, 60])
```

💎 You did it! 10 exercises down — you're on fire! 💎

Pandas

Step 1. Import the necessary libraries

```
In [5]: import pandas as pd
```

Step 2. Import the dataset from this [address](https://raw.githubusercontent.com/justmarkham/DAT8/master/data/users.csv).

```
In [6]: df=pd.read_csv("https://raw.githubusercontent.com/justmarkham/DAT8/master/data/users.csv")
```

Step 3. Assign it to a variable called users and use the 'user_id' as index

```
In [7]: users=pd.read_csv("https://raw.githubusercontent.com/justmarkham/DAT8/master/data/users.csv")
```

Step 4. See the first 25 entries

```
In [ ]: df.head(25)
```

```
Out[ ]:
```

	age	gender	occupation	zip_code
user_id				
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
6	42	M	executive	98101
7	57	M	administrator	91344
8	36	M	administrator	05201
9	29	M	student	01002
10	53	M	lawyer	90703
11	39	F	other	30329
12	28	F	other	06405
13	47	M	educator	29206
14	45	M	scientist	55106
15	49	F	educator	97301
16	21	M	entertainment	10309
17	30	M	programmer	06355
18	35	F	other	37212
19	40	M	librarian	02138
20	42	F	homemaker	95660
21	26	M	writer	30068
22	25	M	writer	40206
23	30	F	artist	48197
24	21	F	artist	94533
25	39	M	engineer	55107

Step 5. See the last 10 entries

```
In [ ]: df.tail(25)
```

```
Out[ ]:      age  gender  occupation  zip_code
```

```
user_id
```

934	61	M	engineer	22902
935	42	M	doctor	66221
936	24	M	other	32789
937	48	M	educator	98072
938	38	F	technician	55038
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

Step 6. What is the number of observations in the dataset?

```
In [8]: users.shape[0]
```

```
Out[8]: 943
```

Step 7. What is the number of columns in the dataset?

```
In [9]: users.shape[1]
```

```
Out[9]: 4
```

Step 8. Print the name of all the columns.

```
In [10]: users.columns
```

```
Out[10]: Index(['age', 'gender', 'occupation', 'zip_code'], dtype='object')
```

Step 9. How is the dataset indexed?

```
In [11]: users.index
```

```
Out[11]: Index([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10,
                ...,
                934, 935, 936, 937, 938, 939, 940, 941, 942, 943],
                dtype='int64', name='user_id', length=943)
```

Step 10. What is the data type of each column?

```
In [12]: users.dtypes
```

```
Out[12]: age          int64
gender         object
occupation     object
zip_code       object
dtype: object
```

Step 11. Print only the occupation column

```
In [13]: users["occupation"]
```

```
Out[13]: user_id
1         technician
2             other
3             writer
4         technician
5             other
...
939        student
940  administrator
941        student
942        librarian
943        student
Name: occupation, Length: 943, dtype: object
```

Step 12. How many different occupations are in this dataset?

```
In [17]: users["occupation"].nunique()
users["occupation"].unique()
```

```
Out[17]: array(['technician', 'other', 'writer', 'executive', 'administrator',
               'student', 'lawyer', 'educator', 'scientist', 'entertainment',
               'programmer', 'librarian', 'homemaker', 'artist', 'engineer',
               'marketing', 'none', 'healthcare', 'retired', 'salesman', 'doctor'],
              dtype=object)
```

Step 13. What is the most frequent occupation?

```
In [21]: users["occupation"].value_counts()
```

```
Out[21]: occupation
student      196
other        105
educator     95
administrator 79
engineer     67
programmer   66
librarian    51
writer       45
executive    32
scientist    31
artist       28
technician   27
marketing    26
entertainment 18
healthcare   16
retired      14
lawyer       12
salesman     12
none         9
homemaker    7
doctor       7
Name: count, dtype: int64
```

```
In [22]: users["occupation"].value_counts().head(1)
```

```
Out[22]: occupation
student    196
Name: count, dtype: int64
```

```
In [23]: users["occupation"].value_counts().idxmax()
```

```
Out[23]: 'student'
```

Step 14. Summarize the DataFrame.

```
In [24]: users.describe()
```

Out[24]:

	age
count	943.000000
mean	34.051962
std	12.192740
min	7.000000
25%	25.000000
50%	31.000000
75%	43.000000
max	73.000000

Step 15. Summarize all the columns

In [25]: `users.describe(include="all")`

Out[25]:

	age	gender	occupation	zip_code
count	943.000000	943	943	943
unique	NaN	2	21	795
top	NaN	M	student	55414
freq	NaN	670	196	9
mean	34.051962	NaN	NaN	NaN
std	12.192740	NaN	NaN	NaN
min	7.000000	NaN	NaN	NaN
25%	25.000000	NaN	NaN	NaN
50%	31.000000	NaN	NaN	NaN
75%	43.000000	NaN	NaN	NaN
max	73.000000	NaN	NaN	NaN

Step 16. Summarize only the occupation column

In [26]: `users["occupation"].describe()`

Out[26]:

count	943
unique	21
top	student
freq	196

Name: occupation, dtype: object

Step 17. What is the mean age of users?

```
In [27]: print(users["age"].mean())
```

```
34.05196182396607
```

Step 18. What is the age with least occurrence?

```
In [28]: users["age"].value_counts().tail()
```

```
Out[28]: age
```

```
7      1
```

```
66     1
```

```
11     1
```

```
10     1
```

```
73     1
```

```
Name: count, dtype: int64
```

You're not just learning, you're mastering it. Keep aiming higher! 

```
In [ ]:
```



Darshan
UNIVERSITY

Data Mining

Lab - 3

Kavyam Joshi | 23010101114 |
16/06/2025

1) First, you need to read the titanic dataset from local disk and display first five records

```
In [5]: import pandas as pd  
import numpy as np
```

```
In [6]: titanic=pd.read_csv("titanic.csv")
```

```
In [7]: titanic.head(5)
```

Out[7]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450

2) Identify Nominal, Ordinal, Binary and Numeric attributes from data sets and display all values.

```
In [8]: Nominal=['Name','Sex','Cabin','Embarked','Ticket']
Ordinal=['Pclass']
Binary=['Sex','Survived']
Numeric=['Age','Fare','SibSp','Parch']
print("Nominal : ", Nominal)
print("Ordinal : ", Ordinal)
print("Binary : ", Binary)
print("Numeric : ", Numeric)
```

```
Nominal : ['Name', 'Sex', 'Cabin', 'Embarked', 'Ticket']
Ordinal : ['Pclass']
Binary : ['Sex', 'Survived']
Numeric : ['Age', 'Fare', 'SibSp', 'Parch']
```

3) Identify symmetric and asymmetric binary attributes from data sets and display all values.

```
In [12]: print("Survived values (Asymmetric values : )")
print(titanic['Survived'].value_counts())
print("-----")
print("Gender count (Symmetric binary : )")
print(titanic['Sex'].value_counts())
```

Survived values (Asymmetric values :)

Survived

0 549

1 342

Name: count, dtype: int64

Gender count (Symmetric binary :)

Sex

male 577

female 314

Name: count, dtype: int64

4) For each quantitative attribute, calculate its average, standard deviation, minimum, mode, range and maximum values.

```
In [21]: quantitative=['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
for column in quantitative:
    print(':::',column,':::')
    print('Mean',titanic[column].mean())
    print('Standard Deviation',titanic[column].std())
    print('Minimun',titanic[column].min())
    print('Maximun',titanic[column].max())
    print('Mode',titanic[column].mode()[0])
    print('Range',titanic[column].max()-titanic[column].min())
    print("-----")
```

```

::: PassengerId ::::
Mean 446.0
Standard Deviation 257.3538420152301
Minimun 1
Maximun 891
Mode 1
Range 890
-----
::: Survived ::::
Mean 0.3838383838383838
Standard Deviation 0.4865924542648585
Minimun 0
Maximun 1
Mode 0
Range 1
-----
::: Pclass ::::
Mean 2.308641975308642
Standard Deviation 0.8360712409770513
Minimun 1
Maximun 3
Mode 3
Range 2
-----
::: Age ::::
Mean 29.69911764705882
Standard Deviation 14.526497332334044
Minimun 0.42
Maximun 80.0
Mode 24.0
Range 79.58
-----
::: SibSp ::::
Mean 0.5230078563411896
Standard Deviation 1.1027434322934275
Minimun 0
Maximun 8
Mode 0
Range 8
-----
::: Parch ::::
Mean 0.38159371492704824
Standard Deviation 0.8060572211299559
Minimun 0
Maximun 6
Mode 0
Range 6
-----
::: Fare ::::
Mean 32.204207968574636
Standard Deviation 49.693428597180905
Minimun 0.0
Maximun 512.3292
Mode 8.05

```

Range 512.3292

6) For the qualitative attribute (class), count the frequency for each of its distinct values.

```
In [17]: print(titanic['Pclass'].value_counts())
```

```
Pclass
3      491
1      216
2      184
Name: count, dtype: int64
```

7) It is also possible to display the summary for all the attributes simultaneously in a table using the describe() function. If an attribute is quantitative, it will display its mean, standard deviation and various quantiles (including minimum, median, and maximum) values. If an attribute is qualitative, it will display its number of unique values and the top (most frequent) values.

```
In [24]: print('Summmmary for passanger of numeric attributes : ')
print(titanic.describe())
print('-----')
print('Summmmary for passanger of all types : ')
print(titanic.describe(include='all'))
print('-----')
print('Qualatitive data')
print(titanic.describe(include='object'))
```

Summary for passanger of numeric attributes :

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Summary for passanger of all types :

	PassengerId	Survived	Pclass	Name	Sex \
count	891.000000	891.000000	891.000000	891	891
unique	NaN	NaN	NaN	891	2
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male
freq	NaN	NaN	NaN	1	577
mean	446.000000	0.383838	2.308642	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN

	Age	SibSp	Parch	Ticket	Fare	Cabin \
count	714.000000	891.000000	891.000000	891	891.000000	204
unique	NaN	NaN	NaN	681	NaN	147
top	NaN	NaN	NaN	347082	NaN	B96 B98
freq	NaN	NaN	NaN	7	NaN	4
mean	29.699118	0.523008	0.381594	NaN	32.204208	NaN
std	14.526497	1.102743	0.806057	NaN	49.693429	NaN
min	0.420000	0.000000	0.000000	NaN	0.000000	NaN
25%	20.125000	0.000000	0.000000	NaN	7.910400	NaN
50%	28.000000	0.000000	0.000000	NaN	14.454200	NaN
75%	38.000000	1.000000	0.000000	NaN	31.000000	NaN
max	80.000000	8.000000	6.000000	NaN	512.329200	NaN

	Embarked
count	889
unique	3
top	S
freq	644
mean	NaN

```
std      NaN
min      NaN
25%     NaN
50%     NaN
75%     NaN
max      NaN
```

Qualitative data

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Braund, Mr. Owen Harris	male	347082	B96 B98	S
freq	1	577	7	4	644

8) For multivariate statistics, you can compute the covariance and correlation between pairs of attributes.

```
In [25]: print("Covariance matrix : ")
print(titanic.cov(numeric_only=True))
```

Covariance matrix :

	PassengerId	Survived	Pclass	Age	SibSp \
PassengerId	66231.000000	-0.626966	-7.561798	138.696504	-16.325843
Survived	-0.626966	0.236772	-0.137703	-0.551296	-0.018954
Pclass	-7.561798	-0.137703	0.699015	-4.496004	0.076599
Age	138.696504	-0.551296	-4.496004	211.019125	-4.163334
SibSp	-16.325843	-0.018954	0.076599	-4.163334	1.216043
Parch	-0.342697	0.032017	0.012429	-2.344191	0.368739
Fare	161.883369	6.221787	-22.830196	73.849030	8.748734

	Parch	Fare
PassengerId	-0.342697	161.883369
Survived	0.032017	6.221787
Pclass	0.012429	-22.830196
Age	-2.344191	73.849030
SibSp	0.368739	8.748734
Parch	0.649728	8.661052
Fare	8.661052	2469.436846

```
In [26]: print("Corelation Matrix : ")
print(titanic.corr(numeric_only=True))
```

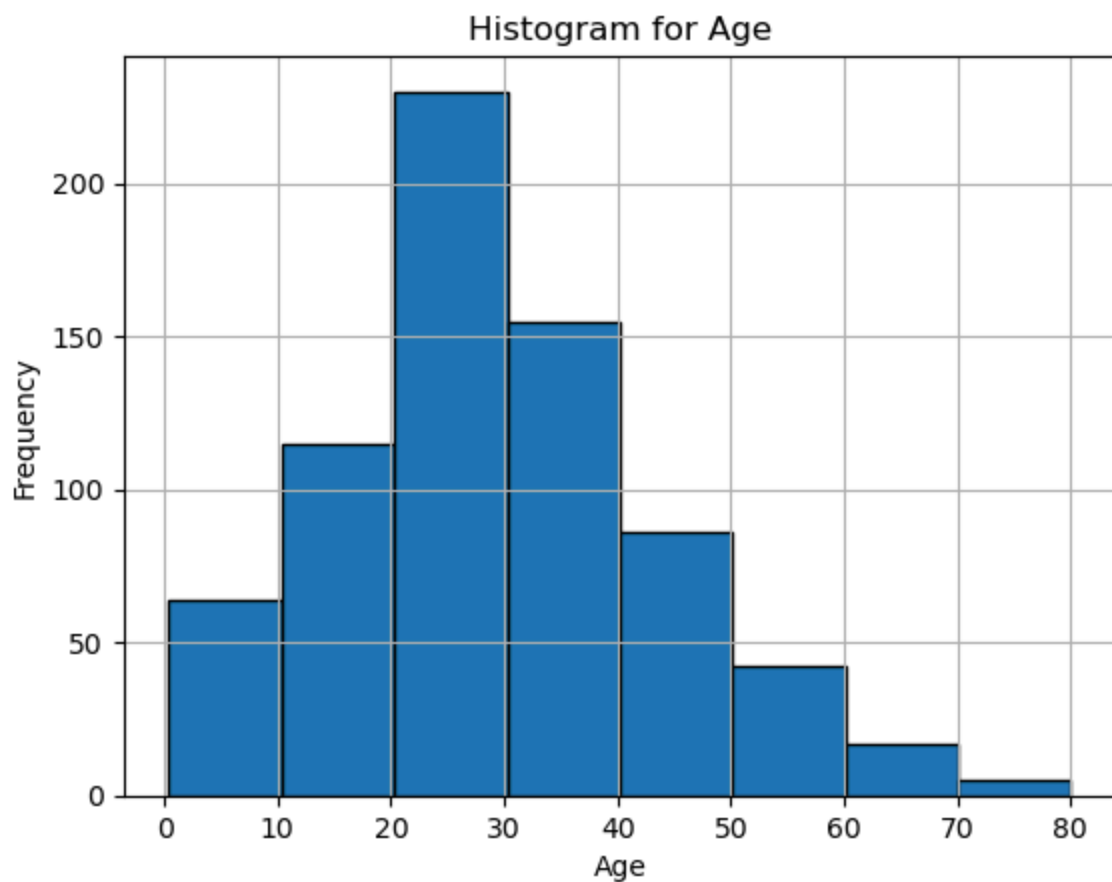

Corelation Matrix :

	PassengerId	Survived	Pclass	Age	SibSp	Parch	\
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	

	Fare
PassengerId	0.012658
Survived	0.257307
Pclass	-0.549500
Age	0.096067
SibSp	0.159651
Parch	0.216225
Fare	1.000000

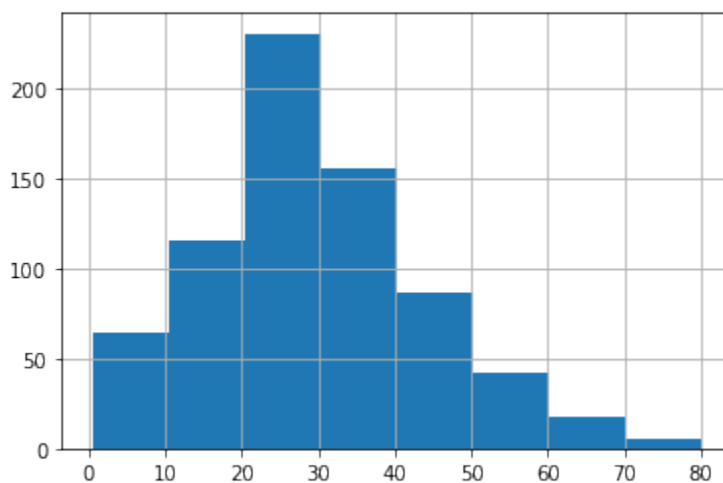
9) Display the histogram for Age attribute by discretizing it into 8 separate bins and counting the frequency for each bin.

```
In [36]: import matplotlib.pyplot as plt
titanic['Age'].dropna().hist(bins=8, edgecolor='k')
plt.title("Histogram for Age")
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



In [13]:

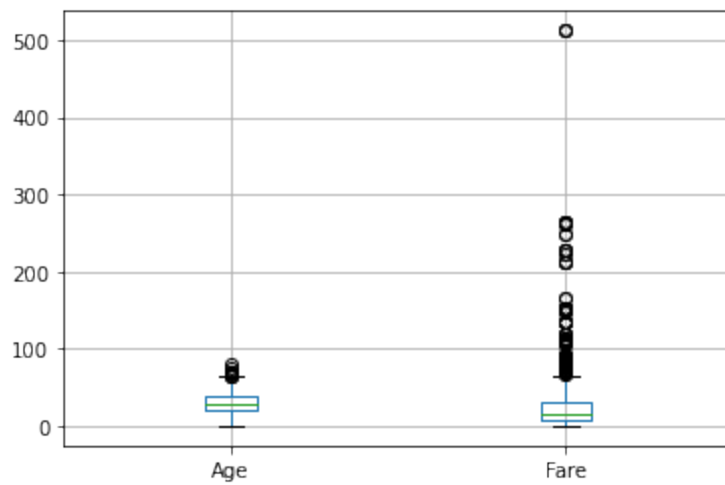
Out[13]: <AxesSubplot:>



10) A boxplot can also be used to show the distribution of values for each attribute.

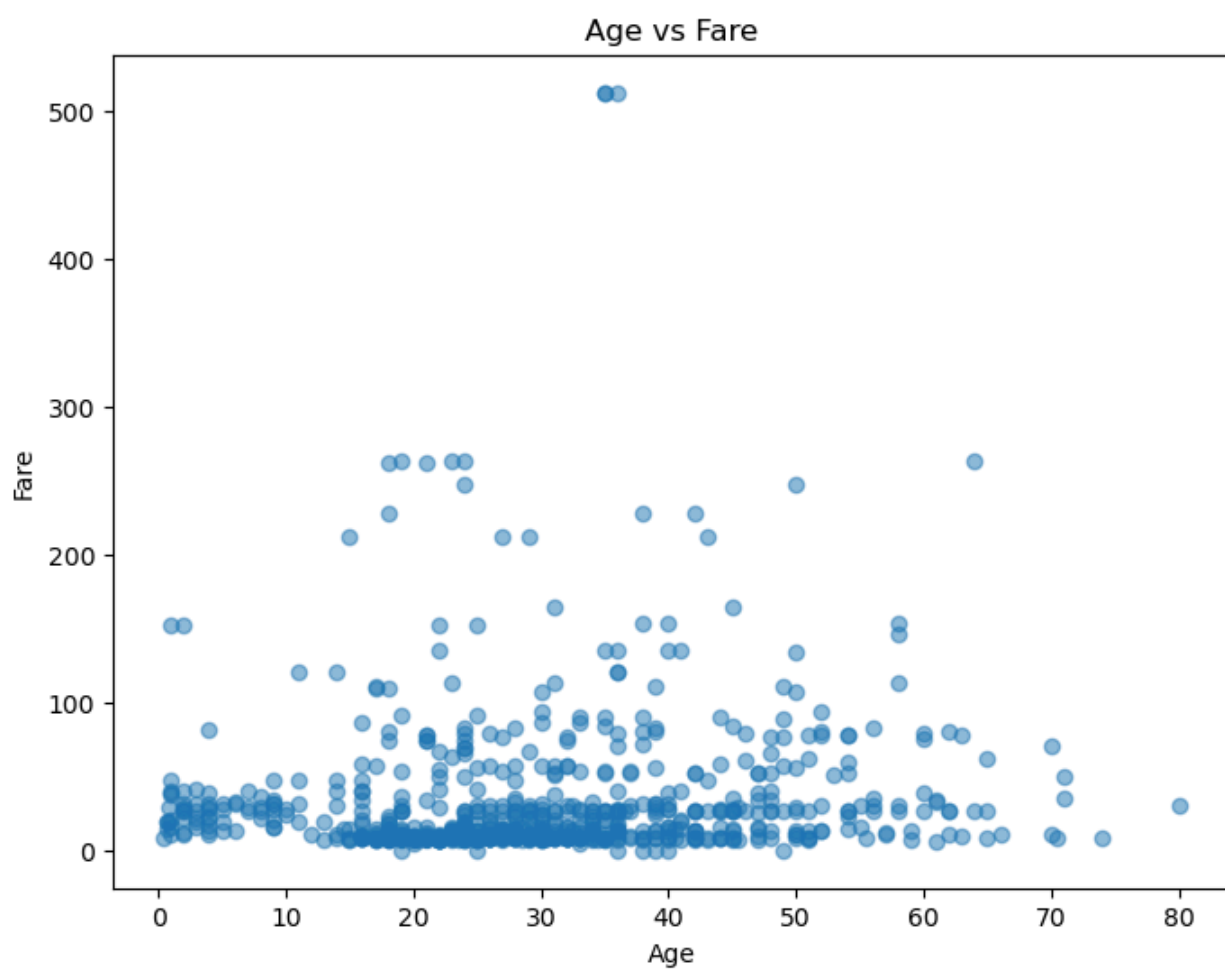
In [17]:

Out[17]: <AxesSubplot:>



11) Display scatter plot for any 5 pair of attributes , we can use a scatter plot to visualize their joint distribution.

```
In [41]: plt.figure(figsize=(8,6))
plt.scatter(titanic['Age'],titanic['Fare'],alpha=0.5)
plt.xlabel('Age')
plt.ylabel('Fare')
plt.title('Age vs Fare')
plt.show()
```



In []:



Darshan
UNIVERSITY

Data Mining

Lab - 4

Kavyam Joshi | 23010101114 |
23/06/2025

Step 1. Import the necessary libraries

```
In [1]: import numpy as np  
import pandas as pd
```

Step 2. Import the dataset from this [address](#).

```
In [2]: url="https://raw.githubusercontent.com/justmarkham/DAT8/master/data/chipotle.t  
chipo=pd.read_csv(url,sep='\t')
```

Step 3. Assign it to a variable called chipo.

```
In [3]: url="https://raw.githubusercontent.com/justmarkham/DAT8/master/data/chipotle.t  
chipo=pd.read_csv(url,sep='\t')
```

Step 4. See the first 10 entries

```
In [4]: chipo.head(10)
```

Out[4]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	\$2.39
1	1	1	Izze	[Clementine]	\$3.39
2	1	1	Nantucket Nectar	[Apple]	\$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	\$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98
5	3	1	Chicken Bowl	[Fresh Tomato Salsa (Mild), [Rice, Cheese, Sou...	\$10.98
6	3	1	Side of Chips	NaN	\$1.69
7	4	1	Steak Burrito	[Tomatillo Red Chili Salsa, [Fajita Vegetables...	\$11.75
8	4	1	Steak Soft Tacos	[Tomatillo Green Chili Salsa, [Pinto Beans, Ch...	\$9.25
9	5	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Pinto...	\$9.25

Step 5. What is the number of observations in the dataset?

In [5]: `# Solution 1`
`chipo.shape[0]`

Out[5]: 4622

In [6]: `# Solution 2`
`chipo.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4622 entries, 0 to 4621
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              4622 non-null   int64
1   quantity              4622 non-null   int64
2   item_name              4622 non-null   object
3   choice_description     3376 non-null   object
4   item_price             4622 non-null   object
dtypes: int64(2), object(3)
memory usage: 180.7+ KB

```

Step 6. What is the number of columns in the dataset?

```
In [14]: chipo.shape[1]
```

```
Out[14]: 5
```

Step 7. Print the name of all the columns.

```
In [17]: chipo.columns
```

```
Out[17]: Index(['order_id', 'quantity', 'item_name', 'choice_description',  
               'item_price'],  
              dtype='object')
```

Step 8. How is the dataset indexed?

```
In [18]: chipo.index
```

```
Out[18]: RangeIndex(start=0, stop=4622, step=1)
```

Step 9. Number of Unique Items ?

```
In [7]: chipo['item_name'].nunique()
```

```
Out[7]: 50
```

Step 10. Which was the most-ordered item?

```
In [20]: c = chipo.groupby('item_name')  
c = c.sum()  
c = c.sort_values(['quantity'],ascending=False)  
c.head(1)
```

```
Out[20]:
```

	order_id	quantity	choice_description	item_price
item_name				
Chicken Bowl	713926	761	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98 \$10.98 \$11.25 \$8.75 \$8.49 \$11.25 \$8.75 ...

Step 11. How many items were orderd in total?

```
In [21]: chipo['quantity'].sum()
```

```
Out[21]: 4972
```

Step 12. Turn the item price into a float

Step 12.a. Check the item price type

```
In [22]: chipo['item_price'].dtype
```

```
Out[22]: dtype('O')
```

Step 12.b. Create a lambda function and change the type of item price

```
In [ ]: dollarizer = lambda x: float(x[1:-1])  
chipo.item_price=chipo.item_price.apply(dollarizer)
```

Step 12.c. Check the item price type

```
In [27]: chipo['item_price'].dtype
```

```
Out[27]: dtype('float64')
```

Step 14. How much was the revenue for the period in the dataset?

```
In [28]: revenue = (chipo['quantity']* chipo['item_price']).sum()  
print('revenue : $'+str(np.round(revenue,2)))
```

```
revenue : $39237.02
```

Step 15. How many orders were made ?

```
In [31]: order = chipo.order_id.value_counts().count()  
order
```

```
Out[31]: 1834
```

Step 17. How many different choice descriptions are there?

```
In [33]: choice_descriptions = chipo['choice_description'].nunique()  
choice_descriptions
```

```
Out[33]: 1043
```


Step 18. What items have been ordered more than 100 times?

```
In [37]: items=chipo.groupby('item_name')['quantity'].sum()  
items[items>100]
```

```
Out[37]: item_name  
Bottled Water          211  
Canned Soda           126  
Canned Soft Drink     351  
Chicken Bowl          761  
Chicken Burrito        591  
Chicken Salad Bowl    123  
Chicken Soft Tacos     120  
Chips                  230  
Chips and Fresh Tomato Salsa 130  
Chips and Guacamole    506  
Side of Chips          110  
Steak Bowl            221  
Steak Burrito          386  
Name: quantity, dtype: int64
```

Step 19. What is the average revenue amount per order?

```
In [40]: # Solution 1  
chipo['revenue'] = chipo['item_price']*chipo['quantity']  
order_grouped = chipo.groupby(by=['order_id']).sum()  
order_grouped['revenue'].mean()
```

```
Out[40]: 21.39423118865867
```

```
In [41]: # Solution 2  
chipo.groupby(by=['order_id']).sum()['revenue'].mean()
```

```
Out[41]: 21.39423118865867
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```



Kavyam Joshi | 23010101114 | 30/06/
2025

Lab - 5 - Data Preprocessing

1) First, you need to read the titanic dataset from local disk and display Last five records

```
In [5]: import pandas as pd  
import numpy as np
```

```
In [4]: df = pd.read_csv('titanic.csv')  
df
```

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

In [6]: `df.tail(5)`

Out[6]:	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	21153
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	11205
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C 660
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	11136
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	37037

2) Handle Missing Values in data set [use dropna(), fillna(), and interpolate]

```
In [12]: # dropna
df.dropna()
```

Out[12]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
	6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	174
	10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	95
	11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	1137
	...	...	...	...	...	...	...	...	...	
	871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	117
	872	873	0	1	Carlsson, Mr. Frans Olof	male	33.0	0	0	6
	879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	117
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113

183 rows × 12 columns

In [13]: `df.dropna(how = 'all')`

Out[13]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

In [14]: `df.dropna(how = 'any')`

Out[14]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
	6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	174
	10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	95
	11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	1137
	...	...	...	...	...	...	...	...	...	
	871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	117
	872	873	0	1	Carlsson, Mr. Frans Olof	male	33.0	0	0	6
	879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	117
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113

183 rows × 12 columns

In [15]: `df.dropna(how = 'any', axis = 1)`

Out[15]:

	PassengerId	Survived	Pclass	Name	Sex	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	1	0	A/5 21171	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1	0	PC 17599	7
2	3	1	3	Heikkinen, Miss. Laina	female	0	0	STON/O2. 3101282	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1	0	113803	5
4	5	0	3	Allen, Mr. William Henry	male	0	0	373450	
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	0	0	211536	1
887	888	1	1	Graham, Miss. Margaret Edith	female	0	0	112053	3
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	1	2	W./C. 6607	2
889	890	1	1	Behr, Mr. Karl Howell	male	0	0	111369	3
890	891	0	3	Dooley, Mr. Patrick	male	0	0	370376	

891 rows × 9 columns

```
In [16]: #fillna
df.fillna(30)
```


Out[16]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	30.0	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

In [25]: `df.fillna({'Age' : 35, 'Cabin' : 'Not Available' })`

Out[25]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	35.0	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

```
In [20]: age_mean = df.Age.mean()
age_mean
```

Out[20]: 29.69911764705882

```
In [21]: age_median = df.Age.median()  
age_median
```

```
Out[21]: 28.0
```

```
In [23]: age_mode = df.Age.mode()  
age_mode
```

```
Out[23]: 0    24.0  
Name: Age, dtype: float64
```

```
In [33]: mean = df.fillna({'Age' : age_mean})  
median = df.fillna({'Age' : age_median})  
mode = df.fillna({'Age' : age_mode})  
print(mean)  
print(median)  
print(mode)
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

	Name	Sex	Age	\
0	Braund, Mr. Owen Harris	male	22.000000	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	
2	Heikkinen, Miss. Laina	female	26.000000	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	
4	Allen, Mr. William Henry	male	35.000000	
..	...	...	...	
886	Montvila, Rev. Juozas	male	27.000000	
887	Graham, Miss. Margaret Edith	female	19.000000	
888	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	
889	Behr, Mr. Karl Howell	male	26.000000	
890	Dooley, Mr. Patrick	male	32.000000	

	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	A/5 21171	7.2500	NaN	S
1	1	0	PC 17599	71.2833	C85	C
2	0	0	STON/O2. 3101282	7.9250	NaN	S
3	1	0	113803	53.1000	C123	S
4	0	0	373450	8.0500	NaN	S
..	...	...	...	...	...	...
886	0	0	211536	13.0000	NaN	S
887	0	0	112053	30.0000	B42	S
888	1	2	W./C. 6607	23.4500	NaN	S
889	0	0	111369	30.0000	C148	C
890	0	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

Name	Sex	Age	SibSp	\
------	-----	-----	-------	---

0		Braund, Mr. Owen Harris	male	22.0	1
1	Cumings, Mrs. John Bradley (Florence Briggs Th...		female	38.0	1
2		Heikkinen, Miss. Laina	female	26.0	0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)		female	35.0	1
4		Allen, Mr. William Henry	male	35.0	0
..		...	...	...	...
886		Montvila, Rev. Juozas	male	27.0	0
887		Graham, Miss. Margaret Edith	female	19.0	0
888	Johnston, Miss. Catherine Helen "Carrie"		female	28.0	1
889		Behr, Mr. Karl Howell	male	26.0	0
890		Dooley, Mr. Patrick	male	32.0	0

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
..	...	...	...	...	...
886	0	211536	13.0000	NaN	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	NaN	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	
..	...	...	...	...	
886	Montvila, Rev. Juozas	male	27.0	0	
887	Graham, Miss. Margaret Edith	female	19.0	0	
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	
889	Behr, Mr. Karl Howell	male	26.0	0	
890	Dooley, Mr. Patrick	male	32.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S

1	0	PC 17599	71.2833	C85	C
2	0	STON/02. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
..	...	...	...	...	...
886	0	211536	13.0000	NaN	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	NaN	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

```
In [11]: #using interpolate
# Option 2: Interpolate only numeric columns
df[df.select_dtypes(include=['number']).columns] = \
    df.select_dtypes(include=['number']).interpolate()
df
```

```
Out[11]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	175
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	3734
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	22.5	1	2	W 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	3703

891 rows × 12 columns

3) Apply Scaling to AGE attribute with min max, decimal scaling and z score.

```
In [56]: fill = df.fillna(df.Age.mean(), inplace = True)
```

```
data1 = df.copy()
minimum = df.Age.min()
print(minimum)
maximum = df.Age.max()
print(maximum)
data1['newAge'] = (data1['Age']-minimum)/(maximum-minimum)
data1
```

0.42

80.0

Out[56]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.000000	0	0
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.000000	0	0
887	888	1	1	Graham, Miss. Margaret Edith	female	19.000000	0	0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.000000	0	0
890	891	0	3	Dooley, Mr. Patrick	male	32.000000	0	0

891 rows × 13 columns

In [58]:

```
data3 = df.copy()
maxAge = df.Age.max()
noOfDigits = len(str(int(maxAge)))
data1['AgeDS'] = data1['Age']/(10**noOfDigits)
data1
```

Out[58]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.000000	0	0
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.000000	0	0
887	888	1	1	Graham, Miss. Margaret Edith	female	19.000000	0	0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.000000	0	0
890	891	0	3	Dooley, Mr. Patrick	male	32.000000	0	0

891 rows × 14 columns

```
In [60]: data4 = df.copy()
data4['Age']=(data4['Age']-data4.Age.mean())/data4.Age.std()
data4
```

Out[60]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	-0.592148	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	0.638430	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	-0.284503	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	0.407697	1	0
4	5	0	3	Allen, Mr. William Henry	male	0.407697	0	0
...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	-0.207592	0	0
887	888	1	1	Graham, Miss. Margaret Edith	female	-0.822881	0	0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	0.000000	1	2
889	890	1	1	Behr, Mr. Karl Howell	male	-0.284503	0	0
890	891	0	3	Dooley, Mr. Patrick	male	0.176964	0	0

891 rows × 12 columns



Darshan
UNIVERSITY

Data Mining

Lab - 6

Kavyam Joshi | 23010101114 |
11/07/2025

Dimensionality Reduction using NumPy

◇ What is Data Reduction?

Data reduction refers to the process of reducing the amount of data that needs to be processed and stored, while preserving the essential patterns in the data.

Why do we reduce data?

- To reduce computational cost.
- To remove noise and redundant features.
- To improve model performance and training time.
- To visualize high-dimensional data in 2D or 3D.

Common data reduction techniques include:

- Principal Component Analysis (PCA)
- Feature selection
- Sampling

❖ What is Principal Component Analysis (PCA)?

PCA is a **dimensionality reduction technique** that transforms a dataset into a new coordinate system. It identifies the **directions (principal components)** where the variance of the data is maximized.

Key Concepts:

- **Principal Components:** New features (linear combinations of original features) capturing most variance.
- **Eigenvectors & Eigenvalues:** Used to compute these principal directions.
- **Covariance Matrix:** Measures how features vary with each other.

PCA helps in **visualizing high-dimensional data**, **noise reduction**, and **speeding up algorithms**.

❖ NumPy Functions Summary for PCA

Function	Purpose
<code>np.mean(X, axis=0)</code>	Compute mean of each column (feature-wise mean).
<code>X - np.mean(X, axis=0)</code>	Centering the data (zero mean).
<code>np.cov(X, rowvar=False)</code>	Compute covariance matrix for features.
<code>np.linalg.eigh(cov_mat)</code>	Get eigenvalues and eigenvectors (for symmetric matrices).
<code>np.argsort(values)[::-1]</code>	Sort values in descending order.
<code>np.dot(X, eigenvectors)</code>	Project original data onto new axes.

Step 1: Load the Iris Dataset

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [8]: iris = pd.read_csv('iris.csv')
iris
```

```
Out[8]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [42]: X = iris.drop(columns = "species")
Y = iris['species'].map({
    'setosa' : 0,
    'versicolor' : 1,
    'virginica' : 2
})
```

```
In [43]: print("Original shape : ",X.shape)
```

Original shape : (150, 4)

Step 2: Standardize the data (zero mean)

```
In [44]: X_meaned = X - np.mean(X,axis = 0)

print("Data after centering (first 5 rows):\n",X_meaned[:5])
```

```
Data after centering (first 5 rows):
   sepal_length  sepal_width  petal_length  petal_width
0    -0.743333    0.442667    -2.358    -0.999333
1    -0.943333   -0.057333    -2.358    -0.999333
2    -1.143333    0.142667    -2.458    -0.999333
3    -1.243333    0.042667    -2.258    -0.999333
4    -0.843333    0.542667    -2.358    -0.999333
```

Step 3: Compute the Covariance Matrix

```
In [34]: cov_mat = np.cov(X_meaned, rowvar = False)
print("Covariance matrix shape : ", cov_mat.shape)
```

Covariance matrix shape : (4, 4)

```
In [35]: print(cov_mat)
[[ 0.68569351 -0.042434  1.27431544  0.51627069]
 [-0.042434   0.18997942 -0.32965638 -0.12163937]
 [ 1.27431544 -0.32965638  3.11627785  1.2956094 ]
 [ 0.51627069 -0.12163937  1.2956094  0.58100626]]
```

Step 4: Compute eigenvalues and eigenvectors

```
In [36]: eigen_value, eigen_vectors = np.linalg.eigh(cov_mat)
print("EigenValue:\n", eigen_value)
print("Eigenvector (first 2):\n", eigen_vectors[:, :2])
```

EigenValue:
[0.02383509 0.0782095 0.24267075 4.22824171]
Eigenvector (first 2):
[[0.31548719 0.58202985]
 [-0.3197231 -0.59791083]
 [-0.47983899 -0.07623608]
 [0.75365743 -0.54583143]]

Step 5: Compute eigenvalues and eigenvectors

```
In [37]: sorted_index = np.argsort(eigen_value)[::-1]
sorted_eigenvalues = eigen_value[sorted_index]
sorted_eigenvectors = eigen_vectors[:, sorted_index]
print(sorted_index)
print(sorted_eigenvalues)
print(sorted_eigenvectors)
```

```
[3 2 1 0]
[4.22824171 0.24267075 0.0782095  0.02383509]
[[-0.36138659  0.65658877  0.58202985  0.31548719]
 [ 0.08452251  0.73016143 -0.59791083 -0.3197231 ]
 [-0.85667061 -0.17337266 -0.07623608 -0.47983899]
 [-0.3582892  -0.07548102 -0.54583143  0.75365743]]
```

Step 6: Select the top k eigenvectors (top 2)

```
In [38]: k = 2
eigenvector_subset = sorted_eigenvectors[:,0:k]
print(eigenvector_subset)

[[-0.36138659  0.65658877]
 [ 0.08452251  0.73016143]
 [-0.85667061 -0.17337266]
 [-0.3582892  -0.07548102]]
```

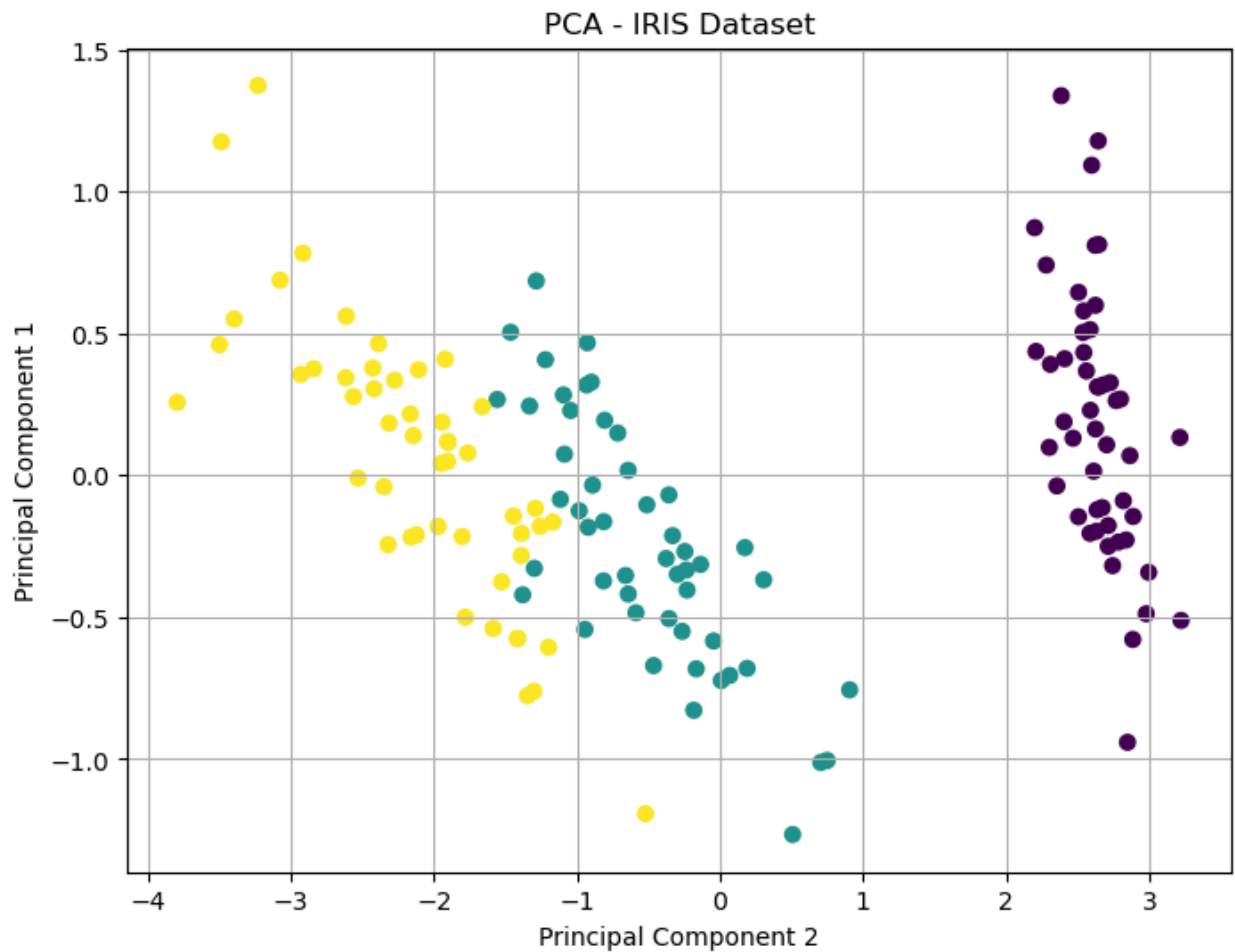
Step 7: Project the data onto the top k eigenvectors

```
In [39]: X_reduced = np.dot(X_meaned , eigenvector_subset)
print("Reduced data shape : ",X_reduced.shape)

Reduced data shape :  (150, 2)
```

Step 8: Plot the PCA-Reduced Data

```
In [45]: plt.figure(figsize = (8,6))
plt.scatter(X_reduced[:,0],X_reduced[:,1],c=Y)
plt.ylabel('Principal Component 1')
plt.xlabel('Principal Component 2')
plt.title('PCA - IRIS Dataset')
plt.grid(True)
plt.show()
```

Extra - Bining Method

5,10,11,13,15,35,50,55,72,92,204,215.

Partition them into three bins by each of the following methods: (a) equal-frequency (equal-depth) partitioning (b) equal-width partitioning

```
In [23]: import numpy as np
#Given data
data = [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]
data.sort() #Sort data
```

Sorted Data: [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]

(a) Equal-Frequency Bins:

Bin 1: [5, 10, 11, 13]

Bin 2: [15, 35, 50, 55]

Bin 3: [72, 92, 204, 215]

(b) Equal-Width Bins:

Bin 1: [5, 10, 11, 13, 15, 35, 50, 55, 72]

Bin 2: [92]

Bin 3: [204, 215]

In []:



Darshan
UNIVERSITY

Data Mining

Lab - 7 (Part 2)

Kavyam Joshi | 23010101114 |
21/07/2025

Step 1: Load the Dataset

Load the `Tdata.csv` file and display the first few rows.

```
In [3]: import pandas as pd

data = pd.read_csv('Tdata.csv')
data
```

```
Out[3]:
```

	Transaction	bread	butter	coffee	eggs	jam	milk
0	T1	1	1	0	0	0	1
1	T2	1	1	0	0	1	0
2	T3	1	0	0	1	0	1
3	T4	1	1	0	0	0	1
4	T5	1	0	1	0	0	0
5	T6	0	0	1	1	1	0

Step 2: Drop the 'Transaction' Column

We're only interested in the items (not the transaction IDs).

```
In [6]: data_items = data.drop(columns=['Transaction'])
data_items
```

```
Out[6]:
```

	bread	butter	coffee	eggs	jam	milk
0	1	1	0	0	0	1
1	1	1	0	0	1	0
2	1	0	0	1	0	1
3	1	1	0	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	1	0

Step 3: Count Single Items

See how many transactions include each item.

```
In [12]: data_items.sum()
```

```
Out[12]: bread      5
butter      3
coffee      2
eggs         2
jam          2
milk         3
dtype: int64
```

Step 4: Define Apriori Function

This function finds frequent itemsets of size 1, 2, and 3 with minimum support.

```
In [39]: from itertools import combinations

def find_frequent_itemsets(data,min_support):
    n = len(data)
    result = []

    for k in [1,2,3]:
        for items in combinations(data.columns,k):
            mask = data[list(items)].all(axis = 1)
            support = mask.sum() / n
            if support >= min_support:
                result.append((frozenset(items),round(support,2)))

    return result
```

Step 5: Run Apriori

Set `min_support = 0.6` and display the frequent itemsets.

```
In [44]: frequent_itemsets = find_frequent_itemsets(data_items, min_support=0.5)
```

Step 6 Display as a DataFrame

```
In [45]: for itemsets, support in frequent_itemsets:
          print(f"{set(itemsets)} -> Support : {support}")
```

```
{'bread'} -> Support : 0.83
{'butter'} -> Support : 0.5
{'milk'} -> Support : 0.5
{'butter', 'bread'} -> Support : 0.5
{'milk', 'bread'} -> Support : 0.5
```

```
In [46]: result_data = pd.DataFrame(frequent_itemsets, columns=['Itemset', 'Support'])
          result_data
```

Out[46]:

	Itemset	Support
--	---------	---------

0	(bread)	0.83
1	(butter)	0.50
2	(milk)	0.50
3	(butter, bread)	0.50
4	(milk, bread)	0.50

Orange Tool : - >Generate Same Frequent Patterns in Orange tools

In []:

Extra : - > Define Apriori Function without itertools

In []:

In []:

Lab-7

1. Apply apriori algorithm on given dataset. Calculate support and confidence then find the association.

TID	Item	Minimum support=2
100	1 3 4	
200	2 3 5	
300	1 2 3 5	
400	2 5	

Scan 0	C ₁	Itemset	Min Sup		L ₁	Itemset	Min. Sup
		{1}	2	scan 0 →		{1}	2
		{2}	3			{2}	3
		{3}	3			{3}	3
	X	{4}	1			{5}	3
		{5}	3				

C ₂	Itemset	Min Sup.	Scan 0	L ₂	Itemset	Min. Sup.
X	{1, 2}	1	→		{1 3}	2
	{1, 3}	2			{2 3}	2
X	{1 5}	1			{2 5}	3
	{2 3}	2			{3 5}	2
	{2 5}	3				
	{3 5}	2				

C ₃	Itemset	Min. Sup.	Scan 0	L ₃	Itemset	Min. Sup.
X	{1 2 3}	1	→		{2 3 5}	2
	{2 3 5}	2				
X	{1 3 5}	1				

Rule Generation:-

Association Rule	Support	Confidence	Confidence (%)
$2^{\wedge}3 \rightarrow 5$	2	$2/2 = 1$	100%
$3^{\wedge}5 \rightarrow 2$	2	$2/2 = 1$	100%
$2^{\wedge}5 \rightarrow 3$	2	$2/3 = 0.66$	66%
$2 \rightarrow 3^{\wedge}5$	2	$2/3 = 0.66$	66%
$3 \rightarrow 2^{\wedge}5$	2	$2/3 = 0.66$	66%
$5 \rightarrow 2^{\wedge}3$	2	$2/3 = 0.66$	66%

2.

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Milk, Diaper, Beer, Cola
5	Bread, Milk, Diaper, Cola

Minimum Support = 2

Here let, Bread=1, Milk=2, Diaper=3, Beer=4, Eggs=5, Cola=6

TID	Items	Scan $\rightarrow$	C_1	Items	Min. Sup.
1	1 2			{1}	3
2	1 3 4 5			{2}	4
3	2 3 4 6			{3}	4
4	2 3 4 6			{4}	3
5	1 2 3 6		X	{5}	1
				{6}	3

L_1	Itemset	Min. Sup.	Scan $\rightarrow$	C_2	Itemset	Min. Sup.
	{1}	3			{1 2}	2
	{2}	4			{1 3}	2
	{3}	4		X	{1 4}	1
	{4}	3		X	{1 6}	1
	{6}	3			{2 3}	3
					{2 4}	2
					{2 6}	3
					{3 4}	3
					{3 6}	3
					{4 6}	2

L ₂	Itemset	Min. Sup.		C ₃	Itemset	Min. Sup.
	{1 2}	2	scan	X	{1 2 3}	1
	{1 3}	2		X	{1 2 4}	0
	{2 3}	3		X	{1 2 6}	1
	{2 4}	2		X	{1 3 4}	1
	{2 6}	3		X	{1 3 6}	1
	{3 4}	3			{2 3 4}	2
	{3 6}	3			{2 3 6}	3
	{4 6}	2			{2 4 6}	2
					{3 4 6}	2

L ₃	Itemset	Min. Sup.		C ₄	Itemset	Min. Sup.
	{2 3 4}	2	scan		{2 3 4 6}	2
	{2 3 6}	3				
	{2 4 6}	2				
	{3 4 6}	2				

Rule Generation:-

Association Rule	Support	Confidence	Confidence (%)
$2 \rightarrow 3 \wedge 4 \wedge 6$	2	$2/4 = 0.5$	50%
$3 \rightarrow 2 \wedge 4 \wedge 6$	2	$2/4 = 0.5$	50%
$4 \rightarrow 2 \wedge 3 \wedge 6$	2	$2/3 = 0.66$	66%
$6 \rightarrow 2 \wedge 3 \wedge 4$	2	$2/3 = 0.66$	66%
$2 \wedge 3 \rightarrow 4 \wedge 6$	2	$2/3 = 0.66$	66%
$2 \wedge 4 \rightarrow 3 \wedge 6$	2	$2/2 = 1$	100%
$2 \wedge 6 \rightarrow 3 \wedge 4$	2	$2/3 = 0.66$	66%
$3 \wedge 4 \rightarrow 2 \wedge 6$	2	$2/3 = 0.66$	66%
$3 \wedge 6 \rightarrow 2 \wedge 4$	2	$2/3 = 0.66$	66%
$4 \wedge 6 \rightarrow 2 \wedge 3$	2	$2/2 = 1$	100%

$$2^4 3^4 \rightarrow 36$$

2

$$2/2 = 1$$

100%

$$2^4 4^6 \rightarrow 3$$

2

$$2/2 = 1$$

100%

$$3^4 4^6 \rightarrow 2$$

2

$$2/2 = 1$$

100%

$$2^4 3^6 \rightarrow 4$$

2

$$2/3 = 0.66$$

66%

Lab-08

Generate the frequent pattern tree for the given dataset.

1. TID	Itemset
1	E K M N O Y
2	D E K N O Y
3	A E K M
4	C K M U Y
5	G E I K O

Minimum Support = 3

Step-1

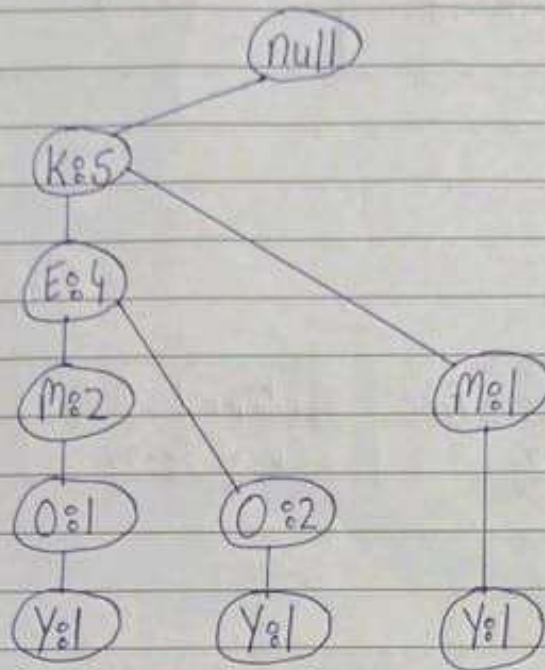
Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

Step-2

{K:5, E:4, O:3, M:3, Y:3}

TID	Sorted Items
1	K E M O Y
2	K E O Y
3	K E M
4	K M Y
5	K E O

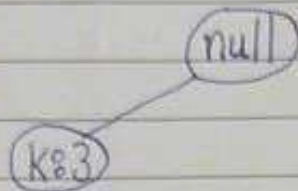
Step-3 Building the FP-tree



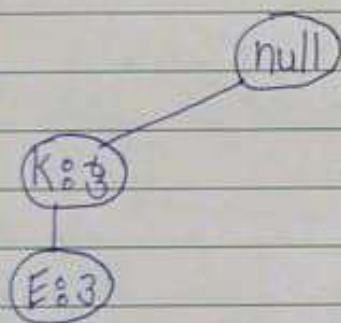
Conditional Pattern Base-

Item	Conditional Pattern Base
Y	{KEMO:1} {KEO:1} {KM:1}
O	{KEM:1} {KE:2}
M	{KE:2} {K:1}
E	{K:4}
K	-

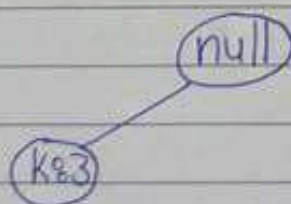
Item	Conditional Pattern Base	Conditional FP tree
Y	$\{KEMO:1\} \{KEO:1\}$ $\{KM:1\}$	$\{K:3\}$



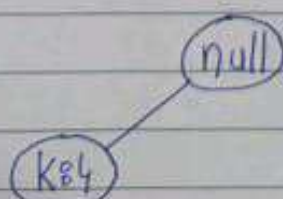
Item	Conditional Pattern Base	Conditional FP tree
O	$\{KEM:1\} \{KE:2\}$	$\{K:3\} \{E:3\}$



Item	Conditional Pattern Base	Conditional FP tree
M	$\{KE:2\} \{K:1\}$	$\{K:3\}$



Item	Conditional Pattern Base	Conditional FP tree
E	$\{K:4\}$	$\{K:4\}$



Item	Conditional Pattern Base	Conditional FP tree	Frequent Patterns Generated
Y	$\{KEMO:1\} \{KEO:1\}$ $\{KEM:1\}$	$\{K:3\}$	$\{K, Y:3\}$
O	$\{KEM:1\} \{KE:2\}$	$\{K:3, E:3\}$	$\{K, O:3\} \{E, O:3\} \{KE, O:3\}$
M	$\{KE:2\} \{K:1\}$	$\{K:3\}$	$\{K, M:3\}$
E	$\{K:4\}$	$\{K:4\}$	$\{KE:4\}$
K	—	—	—

2.	TID	Items	Minimum Support = 2
	1	1 2 5	
	2	2 4	
	3	2 3	
	4	1 2 4	
	5	1 3	
	6	2 3	
	7	1 3	
	8	1 2 3 5	
	9	1 2 3	

Step-1

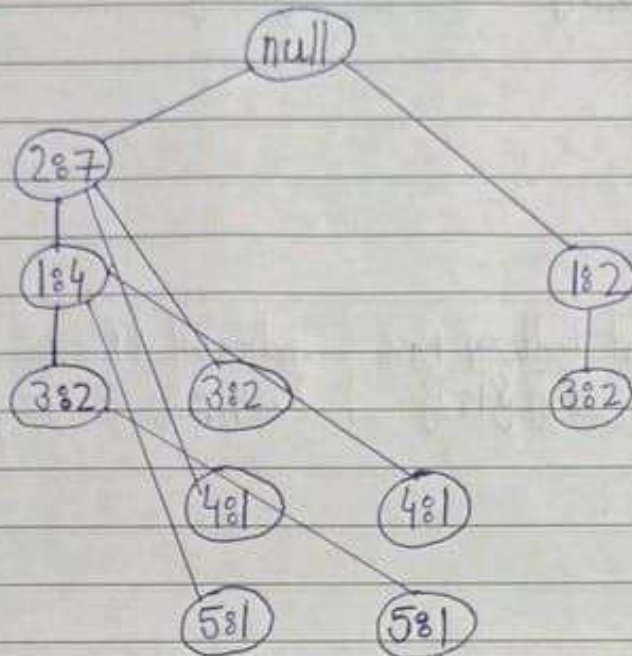
Item	Frequency
1	6
2	7
3	6
4	2
5	2

Step-2

{2:7, 1:6, 3:6, 4:2, 5:2}

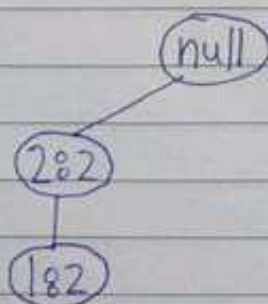
TID	Items
1	2 1 5
2	2 4
3	2 3
4	2 1 4
5	1 3
6	2 3
7	1 3
8	1 2 5
9	2 1 3

Step-3 Building the FP-tree

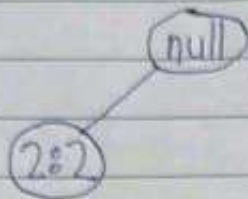


Item	Conditional Pattern Base
5	$\{2:1\} \{2:3:1\}$
4	$\{2:1\} \{2:1\}$
3	$\{2:2\} \{2:2\} \{1:2\}$
1	$\{2:4\}$
2	—

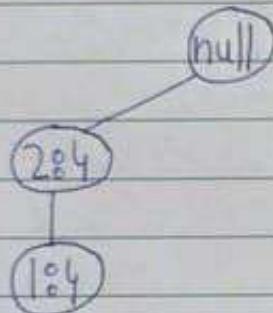
Item	Conditional Pattern Base	Conditional FP tree
5	$\{2:1\} \{2:3:1\}$	$\{2:2, 1:2\}$



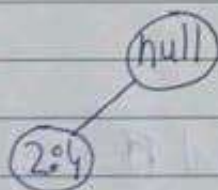
Item	Conditional Pattern Base	Conditional FP tree
4	$\{2:1\} \{2:1\}$	$\{2:2\}$



Item	Conditional Pattern Base	Conditional FP tree
3	$\{2:1, 2:2\} \{2:2\} \{1:2\}$	$\{2:4, 1:2\}$



Item	Conditional Pattern Base	Conditional FP tree
1	$\{2:4\}$	$\{2:4\}$



Item	Conditional Pattern Base	Conditional FP tree	Frequent Pattern Generated
5	$\{2:1\}, \{2:3\}$	$\{2:2\}, \{1:2\}$	$\{2:5\}, \{1:6\}, \{2:5\}$
4	$\{2:1\}, \{2:3\}$	$\{2:2\}$	$\{4:2\}$
3	$\{2:1\}, \{2:2\}, \{1:2\}$	$\{2:4\}, \{1:4\}$	$\{2:3\}, \{2:1\}, \{2:3\}$
2	$\{2:4\}$	$\{2:4\}$	$\{2:2\}$
1	—	—	—

Lab 7, ows* - Orange

File Edit View Widget Window Options Help

CSV File Import

Data

Association Rules

Data

Frequent Itemsets

Frequent Itemsets - Orange

Info

Number of itemsets: 80

Selected itemsets: 1

Selected examples: 2

Expand all Collapse all

Find itemsets

Minimal support: 30%

Max. number of itemsets: 10000

☒ Find Itemsets

Filter itemsets

Contains:

Min. items: 1 Max. items: 2

☒ Apply these filters in search

☒ Send Selection Automatically

Itemsets	Support	%
bread=1	5	83.33
milk=0	2	33.33
milk=1	3	50
coffee=0	4	66.67
milk=1	3	50
eggs=0	4	66.67
milk=0	2	33.33
milk=1	2	33.33
jam=0	4	66.67
milk=1	3	50
butter=0	3	50
jam=0	2	33.33
milk=0	2	33.33
eggs=1	2	33.33
butter=1	3	50
milk=1	2	33.33
milk=0	3	50
milk=1	3	50
coffee=1	2	33.33
milk=0	2	33.33
eggs=1	2	33.33
jam=1	2	33.33
milk=0	2	33.33

Association Rules - Orange

Info

Rules: 42 (shown 42)

Find association rules

Min. supp.: 45 %

Min. conf.: 100 %

Max. rules: 10k

☐ Induce only classification rules

☒ Restrict search by below filters

Find Rules

Filter by Antecedent

Contains:

Items, min: 1 max: 2

Filter by Consequent

Contains:

Items, min: 1 max: 999

☒ Send selection

Supp	Conf	Covr	Strg	Lift	Levr	Antecedent	Consequent
0.500	1.000	0.500	1.667	1.200	0.083	butter=1	bread=1
0.667	1.000	0.667	1.250	1.200	0.111	coffee=0	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	butter=1	coffee=0
0.500	1.000	0.500	1.667	1.200	0.083	butter=1, coffee=0	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	bread=1, butter=1	coffee=0
0.500	1.000	0.500	1.333	1.500	0.167	butter=1	bread=1, coffee=0
0.667	1.000	0.667	1.250	1.200	0.111	eggs=0	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	butter=1	eggs=0
0.500	1.000	0.500	1.667	1.200	0.083	butter=1, eggs=0	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	bread=1, butter=1	eggs=0
0.500	1.000	0.500	1.333	1.500	0.167	butter=1	bread=1, eggs=0
0.500	1.000	0.500	1.667	1.200	0.083	coffee=0, eggs=0	bread=1
0.500	1.000	0.500	1.000	2.000	0.250	butter=1	coffee=0, eggs=0
0.500	1.000	0.500	1.000	2.000	0.250	coffee=0, eggs=0	butter=1
0.500	1.000	0.500	1.333	1.500	0.167	butter=1, eggs=0	coffee=0
0.500	1.000	0.500	1.333	1.500	0.167	butter=1, coffee=0	eggs=0
0.500	1.000	0.500	1.000	2.000	0.250	butter=1	coffee=0, eggs=0
0.500	1.000	0.500	1.000	2.000	0.250	coffee=0, eggs=0	bread=1, butter=1
0.500	1.000	0.500	1.333	1.500	0.167	butter=1, eggs=0	bread=1, coffee=0
0.500	1.000	0.500	1.333	1.500	0.167	butter=1, coffee=0	bread=1, eggs=0
0.500	1.000	0.500	1.000	2.000	0.250	bread=1, butter=1	coffee=0, eggs=0
0.500	1.000	0.500	1.000	2.000	0.250	butter=1	bread=1, coffee=0, eggs=0
0.667	1.000	0.667	1.250	1.200	0.111	jam=0	bread=1
0.500	1.000	0.500	1.667	1.200	0.083	coffee=0, jam=0	bread=1
0.500	1.000	0.500	1.667	1.200	0.083	eggs=0, jam=0	bread=1
0.500	1.000	0.500	1.667	1.200	0.083	milk=1	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	milk=1	coffee=0
0.500	1.000	0.500	1.667	1.200	0.083	coffee=0, milk=1	bread=1
0.500	1.000	0.500	1.333	1.500	0.167	bread=1, milk=1	coffee=0
0.500	1.000	0.500	1.333	1.500	0.167	milk=1	bread=1, coffee=0
0.500	1.000	0.500	1.333	1.500	0.167	milk=1	jam=0